Information Systems Security Engineering (ESSI)
Cybersecurity (CS)
Network Security (SR)
Homework TP#5

NETWORK SECURITY TOOLS HOMEWORK

April 26, 2025

Henrique Santos

Departamento de Sistemas de Informação
Universidade do Minho ©

Contents

1 Fw – Introduction

This work aims to:

- improve knowledge about network security functions, using appropriate tools;
- improve knowledge about technologies and protocols involved with the network perimeter security;
- familiarise students with **iptables** and its operation on **personal firewalls**;
- familiarise students with a tool (**pfSense**) to deploy several network security functions, like firewall and Intrusion Detection;
- develop competencies on the use of the pfSense tool, to filter network traffic and detect intrusions; and
- familiarise students with a virtual environment created for cybersecurity laboratory experiments.

To achieve the above objectives, this exercise is divided into two parts:

- In the first one, we propose a simple exercise involving two VMs linked **by a private network**. A VM deploys a typical server, and that is where the **personal firewall** will be settled through iptables. The other VM is used as a client for testing purposes only.
- In the second one, the virtualisation architecture is expanded to accommodate a **network firewall** (pfSense), forcing some adaptations. Some scenarios are provided, serving as a source of requirements to define traffic filtering rules. Besides, pfSense's modularity is also explored to expand its function into Intrusion Detection functionality. This network firewall also supports the implementation of a DMZ. However, that task is out of the scope of this exercise and is not addressed[1]

1.1 Summary of tasks

1. Preparation of the virtualisation environment (server, client and firewall), giving particular attention to setting an isolated **private network** in the virtual environment. This process is considered trivial, but more details will be provided in the advanced tasks, where multiple network interfaces are required. Anyway, the time devoted to setting up the experience is relevant to developing system administration skills[2].

[1] Even so, the DMZ implementation is a must for the security of actual networks. The pfSense website contains all the information required to implement a DMZ, and it remains a supplementary exercise.

[2] Consider asking for assistance if you are not able to configure the network details

2. Installation (if necessary) and configuration of the required tools and services in the server, to implement a personal firewall. With the recommended version of CentOS (6.09, or 6.10), the effort should be minimal, but if you choose another OS you will have to identify equivalent tools. However, keep in mind that the recommended OS is an EOL (End Of Life) one, so you may need to adjust the package manager to use repositories that are no longer directly available[3]

   NOTE: If you choose CentOS 7, or any other actual OS, it may be configured to use the **firewallD service** by default – for a production system, this service is **much more efficient for firewall management**, with the added cost of making much more obscure the iptables operation, which we would like to avoid in this exercise; therefore, you may have to disable firewallD service to have the system-config-firewall available[a].
   Additionally, if you choose Ubuntu or a related OS, you can find a reference to a tool somehow similar to *system-config-firewall*, named **ufw** (meaning Uncomplicated Firewall); however, despite the name, that tool is much more complicated, and you should avoid it to perform this simple exercise.

   [a] A summary of instructions to carry out this change of configuration is available at https://www.digitalocean.com/community/tutorials/how-to-migrate-from-firewalld-to-iptables-on-centos-7.

3. Test the server's security, provided by the personal firewall (version 1).

4. Tune the personal firewall configuration (version 2) and test again.

5. Adjust the virtual environment to accommodate a network firewall (pfSense), including its installation and configuration.

6. Develop rules for some network contexts and test them (including tools for network traffic generation – **nping**, in particular).

Preliminary note: You should always update the software for safety and performance reasons. This observation also applies to VMs (including the Guest Additions, if the add-on is being used), unless there is some specific indication for not doing so.

2 Tasks – fw (basic)

2.1 Server initial setup

At the server and from a console:

1. Assuming you are using CentOS 6.x, you need to install the FTP service – all other necessary services should be installed already. To do that, you can go to `System` → `Administration`

[3] CentOS community keeps a shadow repository located at http://vault.centos.org instead of the usual location http://mirrorlist.centos.org

→ `Add / Remove Software` and choose, from the FTP server category, expanding the Servers group, "secure FTP – Very Secure FTP Daemon"; then you need to activate and initiate the desired services, which are *httpd*, *sshd*, and *vsftpd*, if they are not running already. To do that you go to `System` → `Administration` → `Services`, select each of the desired services, and first enable it – `enable` – and then start it – `start`). You may also want to configure properly the keyboard and other aspects of the interface at this stage.

> NOTE: operations like installing packages, managing services and most of the commands that deal with iptables require root access privileges. Usually, we do that preceding the command with `sudo`. However, that only works if the actual user is in a special system list referred by *sudoers*, which can only the done by the root user, for which you normally do not know the password. Looks like a dead-end. A quick workaround is using the `sudo su -`, which makes you assume the root role using your password. Not very secure, but effective – assuming you only use it for things like adding the user to the *sudo* group or changing the root password.

After finishing the setup phase, open a terminal and execute the command `netstat -lnt`, which will allow you to check whether the desired services are running (LISTEN) through the respective TCP ports (when using the 'n' option, or names, otherwise); you can also try to open the homepage and access FTP and SSH services, everything in the local host; register in your logbook the details of your architecture – particularly if you use an OS other than CentOS 6.x – the obtained results of the configuration task, and discuss any eventual discrepancies and the workarounds you follow to have the server working as expected.

2. Execute the command `system-config-firewall-tui` (or the equivalent, if you have chosen another OS). This command allows you to set up the firewall in a very simple way via `iptables`.

   You should get a screen like the one shown in Figure 1, which shows the firewall status. In this window (with the `<Tab>` key, or the cursor movement keys, and the `<space>` key for selecting), select the `Enable` option, and then select the `Ok` button, which will end the program execution, after confirmation – depending on the VM you are using the firewall may initially be already enabled, not requiring any modification in this case, since it must end up enabled at this phase.

3. Execute `iptables -L -v`, register its output, and comment:

   (a) the default policies for each of the chains INPUT, FORWARD and OUTPUT;

   (b) the general security level (network security policy) and other information that you can deduce from the obtained output.

4. For security reasons, execute the command `iptables-save> iptables.dump` and keep the file `iptables.dump` (which you should attach to your logbook). If at any stage of the

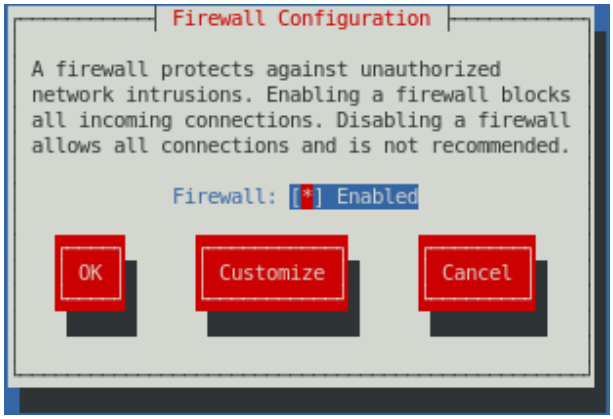

Figure 1: Main interface window of system-config-firewall-tui

exercise you feel lost, you can always come back to this point, cleaning up the corrupted firewall using the `iptables -F` command, and restoring it to this point's state, with the command `iptables-restore < iptables.dump`.

5. Using again the `system-config-firewall-tui` command, disable the firewall and rerun the command `iptables -L -v`

   (a) Register and review the new set of rules, discussing if it is safer or not, and why.

6. Enable the firewall again, reversing the operation performed in the previous step.

2.2 Client initial setup

At the Client side and from a terminal:

1. Start by checking the connectivity by running the command `ping <srv-ip-add>` where `<srv-ip-add>` is, of course, the IP address of the server.

2. Execute the command `nmap -sS <srv-ip-add>`

   (a) What information did you get? Make time to explore some of the additional features of the **nmap** tool (see, for example https://nmap.org/book/port-scanning-tutorial.html).

3. Execute the command `w3m http://<srv-ip-add>`

   NOTE: This command executes a browser that works in text mode, similar to the popular **lynx**, available in other Linux implementations, such as Ubuntu, and it is used the same way; depending on the client you are using, you may have to install one of those tools.

   (a) Are you able to view a page? Register the output in your logbook.

4. Now execute the command `ftp <srv-ip-add>` – you may need to install it tool in the client.

(a) Are you able to establish a connection? Register the output in your logbook.

5. Finally, execute the command `ssh <srv-ip-add>`

> NOTE: if this is the first time you connect to the server using SSH it is natural that you receive a request to accept (or deny) the public certificate from the server. If this happens you must accept the certificate.

(a) Are you able to establish a connection? Register the output in your logbook.

2.3 Fine-tune the personal firewall

Going back to the server:

1. Run the program `system-config-firewall-tui` again. This time select the `Customise` option. Using the arrows and space keys select **FTP**, **WWW** and **SSH** (it may already be selected), which we want to be accessible (trusted). If you want to open a few more services (or ports), you can do it. When finished choose the `Forward` option, which takes you to a window where you can authorise ports that are not directly identified by service names, like in the previous window – no need to change anything at this stage. Continue choosing the `Forward` option, which will take you to the following phases of the configuration:

   (1) the selection of network interfaces you want to have full access (nothing to change);

   (2) network interfaces you want to be masked (nothing to change);

   (3) activation of the port forwarding function (nothing to change);

   (4) the ICMP filter, which lets you select the ICMP commands that the firewall will filter – select all but the `Echo Request (ping)` option;

   (5) finally, you reach the custom rules editor, which let you build specific rules (no need to make changes, also).

   After finishing the configuration phase, you will return to the home window where you will select the `Ok` button and accept the changes, ending the program execution.

2. Execute again the command `iptables -L -v`.

   (a) Register the observed changes and interpret the various rules that have been changed, in the light of the options chosen in the previous step.

   Now, at the client side and using the terminal:

3. Run the command `ping <srv-ip-add>`

   (a) Do you see any changes from previous pinging? Is this what you would expect?

4. Execute the command `w3m http://<srv-ip-add>`

   (a) Are you able to view a page this time? Register the output in your logbook. To quit the program you must press the "q" key.

5. Execute the command `ftp <srv-ip-add>`

   (a) Did you manage to get a connection this time? Register the output in your logbook.

   If you got a connection but cannot log in, that is not strange, since you have not configured vsFTPd server (which you should be using). But perhaps you succeed if you try the user "anonymous", without a password...

6. Finally, execute the command `nmap -sS <srv-ip-add>` again.

   (a) Register the output in your logbook. Compare it with the previously obtained output, and reflect on the current security level.

Going back to the server:

7. Execute the command `ping <cl-ip-add>`, where <cl-ip-add> denotes the client IP address.

   (a) Register the output in your logbook. Is this what you would expect?

8. Execute the command `iptables -L -v` again.

   (a) Register the output in your logbook, and comment the modifications you observe trying to interpret them in light of the activity developed during this task.

2.4 Writing rules

Under normal circumstances, the set of rules is designed and verified offline, and not in an interactive mode, as you just did. A good starting point is the file obtained by the command `iptables-save` (as previously described) and editing it, carefully, adding all the new rules, or deleting the unwanted ones. It is highly recommended to keep track of all changes, with the respective justifications and additional details, since this is a critical operation, and the documentation is essential for maintenance.

Keeping that in mind, the last task of the first exercise aims to prepare an iptables file with a set of rules we typically would like to have in a personal computer firewall, as a baseline protection level.

1. To get an initial template, clear all tables and save the resulting set of rules in a file (you already know the necessary commands).

2. Edit the obtained file, fulfilling the following requirements:

(a) Default policies for all tables should be DROP.

(b) All established or related TCP connections should be accepted, both in INPUT and OUTPUT tables – i.e., after the TCP connection request was performed, all related traffic should be accepted.

(c) All traffic on the loopback interface should be accepted.

(d) Since we are using a default DROP policy, it is necessary to accept outbound DHCP requests, so the workstation can get the network configuration information (IP address, the netmask, and other important information) when booting – required switches: `-p udp --dport 67:68 --sport 67:68`)

(e) Likewise, DNS lookups should also be accepted in the OUTPUT table. DNS can run over TCP or UDP, in both cases targeting port 53. When considering UDP, to match the port number (switch `--dport 53`) it is necessary to precede it with the `-m udp`.

(f) Other fundamental services, or ports, that should be accepted:

- Access to the web (allow outbound HTTP (port 80) and HTTPS (port 443)
- Access to mail relay (allow outbound SMTP, port 25)
- Keeping internal clock synchronized (allow NTP requests over UDP, using port 123 both as source and destination)
- Check external hosts (allow outbound ICMP)

(g) Whenever possible and to make each rule more specific – less chances to get stuck – include the identification of the network interface involved (`-i <iface>` or `-o <iface>` switches).

After finish editing you can test the file loading it with the command `iptables-restore`, debugging eventual errors detected by iptables, and trying to operate the computer, using all involved protocols, until it runs smoothly. Include the final version of the file in your logbook. Execute `iptables -L -v` and register the output. Comment the results you have obtained, along with the details you find relevant to understand them according to the developed firewall rules and the tests you have performed.

3 Tasks – fw (advanced)

3.1 Virtual Lab Architecture

Following the target architecture presented in Figure 2, you need first to configure the virtualisation environment, to allow the virtual machines VM1 and VM2 (which simulate internal computers, clients or servers, like you just used in previous tasks) to connect to a private virtual network (LAN, in the figure), together with the firewall (**pfSense**, VM0 in the figure), which assumes also the gateway function to the outside network (WAN, in the figure). For that purpose, the firewall needs a second network interface to connect to the external network, for which a bridge connection will be appropriate.

Concerning the internal virtual network, none of the usual virtual network modes (NAT and Bridge) allow the type of operation described above, since VMs are typically connected to the host and the hypervisor manages all those functions. Besides, if DHCP is used for LAN, it must be implemented by pfSense and not by the host since it will not be reachable. Virtual Box has a network mode named **Internal Network**, that fulfils those requirements[4]. But other virtualisation platforms also include similar solutions.

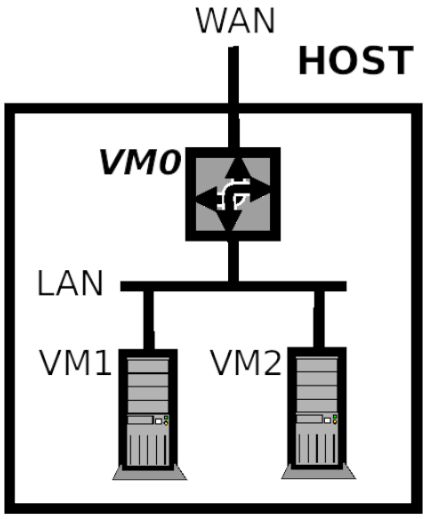

Figure 2: General architecture to accommodate a firewall in a virtual environment

This way, and with VirtualBox, the configuration is very straightforward, being only necessary to follow the next steps.

1. There are several documents in the web covering the pfSense installation (like this one). Anyway, it is important to highlight that since a firewall does not need any peripheral device, and to keep it with the minimal extras principle, when configuring the VM we can safely remove the USB, audio, serial port, and floppy support. Concerning memory and disk space it is better to use the recommendations for hardware requirements.

2. Concerning pfSense VM network configuration, activate Adapter 1 as Bridge (to the host physical network interface), and Adapter 2 as **Internal Network**. Give a name of your choice to the Internal Network (e.g., PrivLAN), and in the Advanced configuration section, chose the type **Paravirtualized Network (virtio-net)**[5], and set the **Promiscuous Mode** to **Allow VMs**. This gives you more options if you need to analyse traffic, even if it is not expected to do. For the configuration required in this exercise you will not need to setup

[4]The VirtualBox documentation includes a very detailed description of this network mode, at https://www.virtualbox.org/manual/ch06.html#network_internal.

[5]With Linux machines, this type of interface has higher performance, since some operations are executed in hardware, in the network interface card itself.

Adaptars 3 and 4, since you only need one internal private network[6].

3. All other VMs (VM1 and VM2 in Figure 2) should have **only one active adapter, configured precisely as Adapter 2 above**. Check those configuration details carefully since an error will compromise the results and may be difficult to detect later.

4. During pfSense installation, the system will ask you to select an interface for the WAN function and another one for the LAN function. In the first case you will choose the one associated to Adapter 1, and in the second case, the one associated to Adapter 2 (you may need to check the MAC addresses in the VirtualBox settings windows, to make the correct choice).

5. After pfSense installation, you will see the IP addresses of each of the network interfaces. The one connected to the WAN will have an IP address provided by an external DHCP server (assuming it is configured as Bridge, as suggested, or even if you decide to configure it as NAT). The one connected to the LAN, by default, will have the IP address 192.168.1.1/24. You may have to change this address, mainly if the WAN network uses the same network address (very common if your host machine is also in a private network) – but you may want to change it by any other reason. The console interface provides you a command to do that – `2) Set interface(s) IP address` –, also allowing to configure IPv6 and DHCP. For the exercise you need to configure DHCP, but not IPv6.

6. After setting up VM1 and VM2 and starting them (it is not relevant which VMs you choose, so let us keep CentOS as a server, and Kali as a workstation), we should test the connectivity between VM1 and VM2, between both and VM0, and also the VM1 and VM2 Internet connectivity.

7. It is possible to configure pfSense from its console (at least partially), but it provides a **remote web-based application** which is much more effective. From any of the other VMs, but preferably from Kali, open the browser, and using the URL http://192.168.1.1 (assuming the default LAN address) we have access to a remote web application to fully configure and manage the firewall. In the first execution, it uses default credentials and will enter a setup wizard, to configure interfaces and the administration account (it is not relevant for this exercise). Anyway, it is a good idea to explore the interface using the official documentation, or any of the tutorials available on the web to install and setup pfSense. It is also a good opportunity to observe the initial rules pfSense includes, through the menu (`Interfaces` → `WAN`) for both LAN and WAN, and **make a prior assessment of the security level, concerning the default settings**.

[6]The tutorials you may find usually describe a complete configuration; dedicate some time to explore the full potential of this tool, to build a more complete Virtual Lab for security engineering applications

8. After being familiar with the remote console, and before entering the core tasks, we need to make a small adjustment. Select `Interfaces` → `WAN`, and scroll down to the last section, **Reserved Networks**. There are two options, both selected by default, which implement two fundamental rules for ordinary network firewalls, on the WAN interface: **Block traffic that comes from private networks and loopback addresses**; and **Block bogon networks**, targeting traffic that uses reserved or not assigned IP addresses (the rules are self-explanatory). However, the first one should be removed since we are running the lab in a virtualized environment, using private addresses. If we keep the rule, we will not be able to access VM1 or VM2 from the WAN, making it impossible to simulate and test external accesses. Unselect that option and press **Save**, following by **Apply Changes**, and selecting `Firewall` → `Rules` check that the respective rule was removed from the WAN section.

Now, you are ready to go.

3.2 Impeding access to a website

Context: after observing how collaborators are using the Internet during labour time, the CEO of your organisation believes that employees are wasting too much time on Facebook. He discusses with you the possibility to modify the Internet Use Policy, including a clause to block access to Facebook. It is your job to enforce that policy. First, you need to determine Facebook's IP address, for which you are going to use `host` and `whois` commands (depending on your environment and location, parameters may need to be adjusted, and results may differ):

```
prompt:~$ host -t a www.facebook.com
www.facebook.com is an alias for star-mini.c10r.facebook.com.
star-mini.c10r.facebook.com has address 157.240.212.35
prompt:~$ whois 157.240.212.35 | grep NetRange
NetRange:       157.240.0.0 - 157.240.255.255
```

In a rule specification, you may need the IP address in the CIDR notation. You can get it using the inetnum interval obtained with command `whois`, eventually using a CIDR calculator (if it is not obvious), such as the one provided at https://www.ipaddressguide.com/cidr – either way, for the above IPs you should obtain **157.240.0.0/16**.

1. Using VM1, or VM2, open the browser and try to access www.facebook.com (it should be working; if not, then you need to debug your firewall configuration).

2. Open the pfSense remote console, and select `Firewall` → `Rules`. Next, move to the LAN section, and choose the ↓ `Add` option (add a rule at the bottom) – this is a very specific rule and should be evaluated after the most generic ones, except those that eventually counteract what we are trying to filter (like default rules), in which case **we may have to move it up**. Configure a rule to **reject TCP** traffic in the **LAN interface**, coming from any machine in the **LAN net**, and going to the **network** you have identified previously (any machine

in the Facebook network). Select also the **Log** option, and give the rule a **description** of your choice. Check if the rule is in the right position, and do not forget to **Save** and **Aply Changes**. Register in your logbook the final configuration.

3. Go back to the browser tab (at the client machine) where you initially opened the Facebook home page, and try to reopen it. You should be unable to connect this time – otherwise the rule is not being verified, you are accessing a page from the browser cache, or there is an alternative route (you can check it with the traceroute command). Go back to the pfSense remote console, refresh the page, and check the **States** indicator of the rule you have created. Register in your logbook the values indicated. Edit the rule, changing the action from **Reject** to **Block**, and try again to access. Did you note any difference? Comment the results.

4. You can also modify the rule to target the specific server instead of all network, and specific ports (80 and 443) instead of any port. Comment on the advantages and limitations of doing so.

5. Select `Status` → `System Logs`, and choose the **Firewall** section. Register in your logbook the evidences of the previous activity. In this window, you also have the possibility of checking the traffic that was blocked, and automatically generate a rule to accept it, or even add the source IP address to a blacklist (a flexible mechanism, but you must use it carefully).

3.3 Add a time schedule to rules

Context: Following the previous decision, you discussed with the CEO the results of the last actions, highlighting the number of complains you received from employees. They are still accessing Facebook from their smartphones and personal devices, getting even more distracted. So, you decided to restrict the application of the previous rule to a limited period, from 10 am to 4 pm, explaining to the collaborators why this is important for the organisation. Now, it is easy to make this modification.

1. You need first to create a schedule, using `Firewall` → `Schedules`, and the **Add** option. It should have a name and **one or more time ranges**. Ranges are defined choosing in a calendar specific days, or weekdays (selecting them on the head of the calendar – in that case, the month has no meaning). Altogether, the schedule is a placeholder, including all time-ranges in which the rule will be applied. We can also add a description to the schedule and the individual ranges. Create a schedule following the specification above, and **Save** it.

2. Select `Firewall` → `Rules`, and edit the rule you create previously. In the **Extra Options** section, select **Display Advanced**. Scroll down until you see the **Schedule** field, and using the pick list choose the created schedule. Save the modifications, and back in the rules window, we will immediately see the modification in the schedule column, with the indication if the rule is being applied at the current time.

Adjust the time range (adding a new one and deleting the other) to a nearby period, so you can test the rule as before, using the browser. Do not forget to register the results in the logbook, including information from logs and a screenshot showing the modified rule.

Note: It is possible to implement this feature in iptables too, using the **module time**.

3.4 Web server public

Context: Most likely, we will need to make the internal HTTP server (VM2) accessible to the outside world, the web – after all, that server was deployed to serve clients using the internet! Since the server is installed in a private network, with a private IP address, we will need to implement a NAT rule to make it accessible. The firewall in the WAN side has the only public available network address. The rule will force to redirect all incoming packets (from the WAN) with TCP port 80 (HTTP), to VM2 (LAN), at the same port. The server will receive a connection request from the firewall (gateway port). The response will be sent to the firewall, which, in turn, will forward it to the remote client. That address translation taking place inside the firewall is what gives the process the name NAT. You will now create such a rule.

1. Select `Firewall` → `NAT`. You are interested in the **Port Forward** section – to explore all possibilities, click on the small button with a question mark, at the upper right side of the window. Now, select one of the Add buttons – assuming there are no rules previously created, it is not relevant which one you use.

2. The necessary fields to setup are:

   Interface where the packets are entering the firewall, which is the WAN interface.

   Protocol you are interested on forwarding TCP packets, only.

   Destination is the IP address the incoming packet should contain. In our case, and since we have only one public address, you must chose the WAN interface IP address.

   Destination port range you are only targeting port 80 (or HTTP) – from and to fields should have the same value in your case. If you choose **Other**, then you need to enter the numeric value in the **Custom** filed.

   Redirected target IP here you must insert the IP address (private address on LAN) of the internal server.

   Redirect target port in your case, the server operates on port 80 also (or HTTP)

   Filter rule association the implementation of a NAT rule requires the creation of one linked rule for WAN interface. If we select **Add associated filter rule**, that rule is created automatically, and that link will be signalled in the NAT rule itself (a cross-arrow line icon). Otherwise, you must create that rule by yourself, which can be a tricky task.

3. Open a browser in your host machine, or any other machine in your external network, and enter the URL `http://<WAN-IP>` (in case you have doubts, the IP addressed is available in the pfSense text console – VM0 –, or in `Status` → `Dashboard`). You should get the home page of the internal server. Register in your logbook the evidences of all your experiments, along with any comment you think necessary to complement your observations.

3.5 Limit the number of connections

Context: Once we have a web server open to the Internet, we start being vulnerable to threats like a denial-of-service (DoS or DDoS – search for additional details in the related documentation) attack. We are particularly aware of the possibility of an attacker making more connection requests than our server is capable of handling. Furthermore, after analysing the server access patterns, we know that in regular operation it is not expected to have **more than twenty connection requests per second**. So, the obvious solution is to prepare a rule in the firewall to block any attempt to violate that value.

1. First, you need to explore a way of testing this type of rule. It is not practical to ask users to make requests at that rate, mainly because it is not possible to control such process. Besides, attackers use software tools to do that, and so the solution is to use also those tools. There are several tools, exploring a broad range of techniques to deploy DoS or DDoS attacks, but for the purpose described you are going to use `nping` (an open source tool, part of the Nmap project, for network traffic generating, and response analysis[7]. If you have Nmap installed in the host computer you already have Nping too. If not, you can install it from the home page, or the package repository for your distribution. From a terminal at the host execute:

   ```
   nping -tcp-connect -p 80 -rate=20 -c 40 <WAN-IP>
   ```

   where WAN-IP is the firewall IP public address. The switches used are self-explanatory, but in short, we are generating 40 (`-c`) TCP connection requests (`-tcp-connect`) to the computer with `<WAN-IP>` address, targeting the port 80 (`-p`), with a of 20 packets per second (`-rate`). The output will show each attempt result, and a final summary similat to:

   ```
   Max rtt: 1.662ms | Min rtt: 0.451ms | Avg rtt: 1.370ms
   TCP connection attempts: 40 | Successful connections: 40 | Failed: 0 (0.00%)
   Nping done: 1 IP address pinged in 1.98 seconds
   ```

   The output shows us the minimum, maximum, and average server response time, the number of responses missed, and the total time (about two seconds, as expected). You can try to raise the number of connection requests per second until you reach a limit. You can also use Wireshark to inspect the traffic generated – these experiments should be registered in

[7] You can get more information at the home page https://nmap.org/nping/

your logbook, and are very helpful to raise your skills with those tools and network traffic, in general.

2. Move now to the pfSense remote console, and Select `Firewall` → `Rules`. In the WAN section locate the NAT rule that forwards traffic to the internal server, and select the edit action. Scroll down until you reach the **Extra Options** section, and select **Display Advanced**, if the Advanced Options are not visible. There are two related fields that you need to setup in this section: i) **Max. src. conn. Rate**, which specifies the maximum number of connection requests allowed, per host; and ii) **Max. src. conn. Rates**, which specifies the time period that applies to the previous parameter (default is one second). Together, these parameters define the rate limit expressed in connections per second – note that the condition applies only to TCP packets. Fill in those fields according to the specified requirements (maximum of twenty connections per second). After saving and applying changes, you will get back to the rules screen, where you can note a new symbol (small sprocket) indicating the rule has extra options (if you pass the pointer over the symbol a popup box appears showing you the settled parameters). Include in your logbook a screen capture of that detail.

3. From the host, rerun the previous `nping` command, but this time raising the rate and packets count (e.g., `-rate=30 -c 30`). The output now should evidence that some connections were not fulfilled, as expected. Now, selecting `Status` → `System Logs` and the **Firewall** section, there should be some entrances showing the refused connections, by a Rule named **virusprot overload table (1000000400)**. In practice, pfSense flagged the source IP address of the host and put it in an internal table used to store forbidden IP addresses that violated the connection count conditions (whatever they are). We can check it selecting `Diagnostics` → `Tables` and choose **virusprot** table from the drop-down list. There should be only one entry, which will remain there for one hour after the last connection attempt is registered (unless you delete it before). Again, include in your logbook all the evidences demonstrating your success in this task.

4. Next, run the same `nping` command from the other VM, first without exceeding the limit imposed, and after with overloading values (e.g., `-rate=40 -c 80`). Register in your logbook the output and comment the results adding pieces of evidence from logs and tables, as before.

The previous exercise did not cover all possible scenarios of interest, and, definitely, not all pfSense features. In particular, the monitoring operations allowed by **Status** and **Diagnostics** menus are essential for a regular surveillance operation, which is a primary job of a Cybersecurity Operator. The **Dashboard**, as an entry point, can be configured to provide excellent first-view indicators of network problems. The small '+' button in the upper right corner allows choosing several **widgets**, like the **Traffic Graph**, or the **Firewall Logs**. Of course, a real environment has nothing to do with the lab one, and that is the reason no related task was proposed. Even

so, you should be now capable of developing your skills concerning firewalls (both at personal and network levels).

A final remark concerning the rule verification process: along with the exercise, you may have noticed some error indications when you tried to set up some parameters that a specific rule does not accept. This is a feature most firewalls provide for what we can call a basic check. However, keeping a set of rules coherent is more laborious and requires a more robust understanding of how firewalls work and network traffic behaves. This is, and always will be, a demanding research area.

4 NIDS - introduction

Network Intrusion Detection

For this exercise you will use Suricata, but with the firm conviction that the skills developed will be useful to work with other NIDS tools. Besides, Suricata is available in pfSense, as an external module, being very easy to put it to work, and located at the border of the network side-by-side with the firewall, which is a recommended position to implement a NIDS. Nevertheless, using other alternatives should not be difficult. So, you will start with the architecture previously used in the firewall exercise in the proposed virtual lab (see section 3.1).

4.1 Summary of tasks

1. Set up and test Suricata, within pfSense, giving particular attention to the rule set selection and preparation.
2. Set up a MITM attack and test the detection capacity of Suricata. For this task, you will need all the virtual machines working and a tool named **Ettercap** available in the attacker machine (Kali already have it installed).
3. Create rules for a specific purpose, exploring the ambiguity that is typically included in such rules.
4. Review the alerts produced by Suricata and explore some of the tricks to minimise false positives.

5 Tasks – NIDS (basic)

5.1 Setup Suricata

Suricata can be downloaded in binary format for almost all OSs, in source code, integrated with network security platforms (such as Security Onion, or OSSIM), or as an external pfSense package. Since we already have an infrastructure with pfSense in our virtual lab, from the

previous exercise, we will take the last option – follow the instructions in the first task of section 3.1, if it is necessary to install pfSense, or follow the official documentation, when deciding to implement a dedicated box. However, with the chosen strategy, you may need to adjust the resources allocated to pfSense, following Suricata requirements: a minimum of 4GBytes of memory, and two or more processors, depending on the host available, to take advantage of the Suricata multi-threading capabilities.

1. Starting with the pfSense web interface, and selecting `System` → `Package Manager`, you enter a page with the list of installed packages (none, by default). In the tab `Availabe Packages` you have access to a long list of packages, including Suricata. Search for it – taking the opportunity to look at the list of available packages (consult the help page if necessary) – and press the associated `+ Install` button. The process is fully automatic, installing Suricata and Barnyard.

   Note: Barnyard is spooler-like utility used by Snort and Suricata to speed up the alert registering process, implementing the interface with the database.

2. The next step consist on performing a basic configuration, setting up the essential options, but leaving many others with default values, which usually are adequate – the modification of some options requires a deep understanding of the Suricata architecture, demanding additional effort. Expanding the `Service` menu at the pfSense web interface, there should be now an entrance for `Suricata`. Selecting it will open the configuration page, starting with the **Interfaces** section (it should be empty, initially).

   Going now to the `Global Settings` tab you will be able to configure one of the most important components, **the rules to be used**, which is accomplished by the following steps:

   (a) There are several options, and some of them are related to paid services – the business case of IDSs is mostly supported on the rules providing process, behind which there is a lot of continuous research work. Fortunately, there are also some free versions, mainly supported for testing and to demonstrate how an IDS works. You will use the free versions (**ETOpen** and **Snort Community Ruleset**), of course, but it is useful to give a look at the web pages of the service providers and, eventually, to register with Snort VRT (no payment required) to have access to the Free Registered User rules.

   (b) It may be also important to configure the rules **Update Interval**. In a production system this can be critical, but in this exercise one week will be enough.

   (c) Selecting the **Live Rule Swap** capability allows to automatically restart Suricata after a rule update – you will leave it unselected for now.

   (d) The **GeoIP DB Update** feature (selected by default) is also useful.

(e) Finally, you can configure the time interval to keep hosts blocked, if such an intrusion response is being used (fifteen minutes would be a good choice, with a prototype deployment).

Some of the features just described may be defined by a Security Policy, and in a production system it must be set according. After finishing this phase you must press the `Save` button.

3. After configuring the rule sets, you need to update the local stored rules, selecting the `Updates` tab. There is a list of installed rule sets, the date of the last update, and their signatures (it should not be ready at this stage). Pressing the `✓Update` button will do the required job, and after a short time, the list will show the final result. You can also check for any problems during the update process by viewing the management rule sets log - by pressing the `View` button.

4. Next, you will enter the `Interfaces` section, and proceed by adding an interface, using the `+ Add` button. The first option is the interface name, which Suricata heritages from pfSense, and assuming the configuration from the previous exercise, you will have two possible interfaces: WAN, and LAN. This also means that we can monitor any or all the networks created with pfSense (even when you use more then one internal networks). You will start with WAN selected. However, the choice is not relevant, and it could be LAN, as well. Now begins the more complex job, which is the rules settings and fine-tuning.

> Note: you are leaving all other settings with default values, but there are three groups deserving particular attention: i) the **Logging Settings**, which allows defining what Suricata will be logging, in which formats, and where, under assumption that performance and storage space are being taken carefully (e.g., when you decide to send alerts to the system log, the system should be prepared to store much more logs and in a persistent way). Enabling JSON log may also be interesting, in particular when looking for integration with other tools, like the visualisation ones; ii) the `Alert and Blocking Settings`, with the option `Block Offenders`, which, when checked, will force Suricata to block any IP address that generates an alert (the blocking time was previously configured in the Global Settings section); and iii) the **Performance and Detection Engine Settings**, which allows controlling important parameters, with implications on available resources – you may change the `Detect-Engine Profile` to high, when using a powerful host.

(a) Selecting the `Categories` tab will take you to the section were rule sets can be chosen. Scrolling down will show the rule sets from Snort Community and ETOpen repositories, previously selected. At this point, you do not have any criteria to chose specific rule sets, so you are going to press `Select All`.

You will also check the `Resolve Flowbits` option. This is an interesting feature forcing linked rules (by *flowbits*) to be automatically selected. The *flowbits* mechanism imple-

mented by Suricata is part of the rule construction language, allowing to set a named *flowbit* in a rule, which will be tested in another rule(s) – this is particularly useful with TCP or application protocols, where an alert must only be issued after some preconditions are met, imposed by previous packets, helping to reduce false positives or redundant alerts. After setting all the details, we need to `Save` the configuration.

(b) Now you will go to the `Rules` section, where you can control the operating rules, from each rule set. The first rule set is already selected and scrolling down will show the respective list of rules, along with the information about the activation state (see the legend above the list), its action, the unique identification number (SID), the main components of a rule (IP addresses and ports), and the messages logged when a match occurs.

> Note: from the Categories section, if you click on a rule set, you will jump directly to this page, with that rule set already selected.

You can change the state, enabling or disabling each rule, and you can see the rule itself clicking over the SID. Selecting **Active Rules** in the category box will show you that, in the present configuration, you have more than twenty thousand rules, showing the necessity of fine-tuning, otherwise you will get a large number of false positives. Again, you have no criteria, at this point, to unselect any of the rules, of any of the rule sets, so you will leave the list unchanged. After finishing the modifications, you need to click on the `Apply` button, to make modifications effective in Suricata.

(c) You will conclude the interface configuration with the `IP Rep` section, concerning the IP Reputation feature, which you will disable for now. Suricata implements an open architecture strategy concerning IP reputation, where a central server interacts with sensors to manage the reputation lists. The server can use globally accessibly reputation lists or implement its own strategy. Without that infrastructure, Suricata will keep feeding the internal lists using the local alerts information, only.

(d) The other fine tuning options within the `Interfaces` section will not be addressed now, but it may be worth to look at the Variables section, where we can define variables with internal machine names that Suricata will use to help interpretation of alerts and logs.

5. After finishing the configuration, it is now necessary to **start the interface monitoring function**. This is accomplished through the `Services / Suricata / Interfaces` page, pressing the icon with a small arrow under the `Suricata Status` column. Another useful operation you can do now is duplicate the configuration just performed, for another interface. Pressing the middle button (two small overlapped rectangles) under the `Actions` column, will open the initial interface configuration page, already with the LAN interface selected (in case there are only two, and the WAN was the one previously used) and fully configured with the same rules enabled, only requiring to press the `Save` button. After that, you can start the monitoring operation on the second interface, too. Note that if there are not enough

memory, Suricata will give an error – if that happens, it is required to shutdown pfSense and modify the VM memory size. After starting the interfaces, the icons under the `Suricata Status` change, showing a green circle with a check mark indicating the running status, and two more buttons, one to **restart** the service and another to **stop** it.

5.2 Testing and fine tuning Suricata

After starting the monitoring service, Suricata will probably begin showing some alerts immediately, depending on the activity of your network, including the host and the external network in use. Thus, the next steps can produce slightly different results in your case.

1. **Managing alerts** is accomplished through the `Suricata / Service / Alerts` page, where it is possible to visualise alerts originated by each interface, selecting it in the `Instance to View` box – all the elements in the page are considered self-explanatory. In principle, the LAN traffic is residual and will not generate any alerts until we force some activity. The WAN interface has a different behaviour. VMs may access the Internet as part of updating processes, or general information access services, like the pfSense web application itself – internal modules of the web interface access external DNSs, which triggers a rule since internal machines are supposed to access the DNS only through the gateway. Anyway, we can force some activity, e.g., executing the update command in the CentOS machine (`sudo yum update`, or equivalent in one of the VMs), while watching the `Alerts` page (it is not necessary to perform the update, but just forcing the search for updates).

   (a) Register in your logbook the different alerts you get and try to relate them with your system's activity (the above description may help you).

   (b) Take the description of each alert type and perform a google search. Do those alerts correspond to real threats?

2. Even at a small scale, it is evident that the number of alerts can increase rapidly, in a 'noisy' way, making it difficult to analyse them efficiently. Fortunately, there are some easy options to help fine tune Suricata, from the `Alerts` page itself:

   (a) In each line describing an alert, there are three action icons, namely, i) a **small magnifying glass**, allowing you to perform a reverse name resolution operation, for both the source and destination IP addresses – naturally, it only gives useful information for public IP addresses, not private ones – (try it and register the result in your logbook); ii) a **small red 'x'** in GID:SID column, allowing you to disable the rule and remove it from the current rule set – you can always re-enable it later; and iii) a **small** ⊞, allowing to add a rule to a **suppress list**, with no conditions (when pressing it in the GID:SID column), or conditioned by an IP address (when pressing it in the Src or Dst columns) – Suppress lists are used to register rules that we want to stop generating alerts, but that will not

be disable from the rule set (particularly useful when we want to stop alerts for a rule, associated to a particular IP address).

The suppress mechanism has some details that need to be highlighted. When suppressing a rule, Suricata creates automatically a suppress list, adds the rule to it, and links it to the interface. If more rules are suppressed the same way, they will be added to the same list. Suppressed lists can be managed through the page `Services / Suricata / Suppress`, where you can edit, delete, or create lists (hand made), and jump to the interface where a list is first instantiated (small ▶ boxed icon in the `Actions` column).

To gain some experience with this fine-tuning mechanism:

i. from the alert page create a suppress list from the rule whose description includes "ET POLICY GNU/Linux YUM...", **tracked by the source IP address** – using the ⊞ button – you can choose another one if this alert is not present;

ii. next, do the same with the rule whose description includes "ET TOR Known Tor...", but this time **tracked by the destination address** – you can choose another one if this alert is not present;

iii. Go to the `Suppress` page and press the edit option (small pencil icon) within the Actions column, for the list just created (there should be only one); take a screenshot of the list and add it to your logbook; dedicate some time interpreting the way Suricata defines the content of a list, and see the examples at the bottom of the `Suppression List Content` block, which present some of the usual constructs used (very important when creating entries manually, for particular purposes not related to the automatic actions from alerts page);

iv. going back to the `Suppress` page, click on the small arrow icon in the most right position, within the Action column, which will take you to the edit interface setting page of the WAN interface; scroll down to the `Alert Suppressing and Filtering` block, where you can see the suppress list just created attached to the interface; the drop-down button allows you to attach another list or the default list – **if a list is attached to an interface it can not be removed, in the Suppress page**;

v. edit the list removing the second entry (keeping the one related with the package management activity) and save it; go to the `Alerts` page and check that there is only one rule in the suppress list (the one with a **circled 'i'** icon replacing the ⊞); copy to your logbook an evidence of the modification; and

vi. in the CentOS VM execute the update command (or any other activity associated to the alert you have chosen) again and verify the generated alerts, to confirm the excluded one (if there were no errors).

Rationale: The excluded alert was marked as a "Potential Corporate Privacy Violation", assuming there was such a policy stating, for instances, that regular desktop computers cannot perform updates. If the target machine was a server, for which it is expected to have daily updates, the suppression was adequate to remove that false positive.

(b) **Filtering alerts display** can be accomplished from the Alerts page, pressing the '+' **icon** in the `Alert Log View Filter` bar. Not surprisingly, it is possible to filter alerts by identification, IP addresses, protocols, data, description, and classification (all relevant pieces of information related to alerts). That allows viewing just the alerts satisfying the conjunction of parameters indicated, being useful to see, for instances, the number of alerts in a specific time interval, the alerts associated with a particular host, or the number of alerts with the same identification. Notwithstanding the usefulness of this feature, the analysis capacity is limited – **external tools are necessary for improved visualisation and analysis**.

The ultimate goal of previous operations is to identify rules, or rule sets that **you can safely disable or suppress**. That is far from being an easy or quick task, requiring a lot of effort, Google searches, and sharing experiences through related forums, like the one used by pfSense users, at Netgate[8]. As a starting point, but always recognising that there is no single solution, as there are no two identical environments and similar risk-aware perceptions, the following guides can be useful:

i. **Informative alerts** (non suspicious traffic) are raised by traffic variants that can be linked to network components particularities and after evaluated, can be safely disabled;

ii. **Classification** and **priority** are two important details to capture the nature of the alert and the severity level. They are both defined in a file named `classification.config` (within the pfSense implementation it is located at the `/usr/local/etc/suricata` directory, and you can access it through a terminal in the pfSense VM, or remotely through **ssh**, after enabling it). Priority can be any number between 1 and 255, but in the default classification file only values from 1 to 4 are used – higher values mean higher priority.

As an example, in our case, there is probably a large number of alerts classified as "Potentially Bad Traffic", priority 2, resulting from traffic from the pfSense WAN interface, directed to an external name server (port 53) – you can check it using the reverse name function –, with SID 2013743, and a description indicating a query to a suspicious no-IP domain. This traffic is generated by the pfSense web application, and it can be safely ignored. Instead of disabling the rule, which we want active for other occurrences, you can suppress it when involving the WAN interface. After doing that, you can close and start the web application again and check if you are

[8] Available at https://forum.netgate.com/category/53/ids-ips

still receiving those alerts. After suppressing this rule and the one related to updates (previously described), you should have now a much more quiet IDS – it may also be useful to clean the alerts with the `Clear` button, eventually saving them first with the `Download` button.

iii. Alerts related to internal IP addresses and associated with services we trust suggest that the rules are not adjusted. Depending on the situation you can again suppress the rule, or even disable it, or edit the rule to adjust it.

iv. Alerts related to **services (TCP or UDP ports) we are not implementing are irrelevant** and the associated rules can be removed. After all, even if this traffic is malicious, it will not cause any harm since no machine will receive it. You may be missing the opportunity to identify an attacker running a scanning, but the cost of false positives is higher.

v. Not all rule sets previously selected are relevant, and some of them will never be used. If possible, a very effective strategy is disabling those unnecessary (or even undesired). Furthermore, you have selected both ET Open Rules and Snort Community Rules, which have a lot of common or very similar rules (despite, by default, several rules are already disabled). ET Open Rules are frequently considered enough, and from it you may find essential the following rule sets (to which it is necessary to add the rule sets associated with specific services):

- emerging-attack_response.rules
- emerging-bootcc_portgrouped.rules
- emerging-bootcc.rules
- emerging-ciarmy.rules
- emerging-compromised.rules
- emerging-current_events.rules
- emerging-dos.rules
- emerging-dshield.rules
- emerging-exploit.rules
- emerging-malware.rules
- emerging-scan.rules
- emerging-shellcode.rules
- emerging-trojan.rules
- emerging-worm.rules

Overall, fine-tuning an IDS is a continuous and challenging task, and any organization immensely appreciates those mastering the skills necessary to engineering an efficient rule set.

6 Tasks – NIDS (advanced)

6.1 Using scanning tools to test Suricata

In this phase, you will start forcing some traffic that, despite not being offensive, is not benign either. In large, that traffic is linked to network scanning operations, through which it is possible to detect active hosts, active services within hosts, and even to identify versions of applications and OSs. Nmap is one of the most popular tools available for that purpose and the one you will use. But before, it is better to configure the pfSense dashboard (`Status` menu) to include the usual **System Information** and **Interfaces** boards, and, at the right side, the **Traffic Graphs** and the **Security Alerts** boards (this last one should be configured to show at least ten alerts). Optionally, you can hide the WAN information since we are going to work mainly on the LAN, generating the activity with the Kali VM, and targeting the second VM on the local virtual network.

1. From a terminal in Kali, and while visualising the traffic activity and the Suricata alerts in the pfSense dashboard in the background, execute

   `nmap -PS -v <LAN-add>`

   where `<LAN-add>` specifies the LAN address in CIDR notation, and the `-PS` switch indicates the type of scan. With that command, Nmap will "ping" all hosts, using TCP SYN packets using the most usual ports. Observe the output, the generated traffic volume pattern and, in particular, if Suricata produced any alert. If not, should it have? To look for an answer, you need carefully investigate the `emerging-scan.rules` file, already referred, and prepared to detect that type of activity. It can be accessed with any text editor, or through pfSense's Suricata service menu, as explained previously. If you find a rule that you think should generate an alert with the above command, indicate which modifications you must perform for that purpose.

   Try with other options of the Nmap `-P` switch (see the help output) and compare/comment the results.

2. Next, following the same steps, execute the command

   `nmap -sS -v <srv-ip-add>`

   where `<srv-ip-add>` specifies the IP address of the target server, and `-sS` selects a port scan also based on TCP SYN packets and using a standard range of TCP ports. Proceed the analysis of the result as above, including the investigation of the rules, and the use of alternative Nmap `-s` switches (in particular the `-sX`, generally referred as more intrusive). Register all the experiences in your logbook and compare them with the previous process. From the attacker's point of view, which one seems more effective?

To go a deep further with the analysis, you can complement it observing the generated traffic with Wireshark.

3. To complete this phase, execute the command

   `nmap -A -v <srv-ip-add>`

   where the `-A` switch enables Nmap to perform a full scan on the target, including services and versions. This operation is more intrusive and performed at the services level and not at the network level. This time you should receive some alerts. Take note of those alerts and try to justify them with the type of action performed with Nmap.

Another useful tool to test an IDS is **Hping** which is similar to the well-known ping, but allowing to use many other protocols and features that can be used to simulate some attacks. Kali includes the last version, `hping3`, with a useful description available at this link.

4. Keeping the same screen organisation (i.e., with the pfSense dashboard in the background, showing the traffic graphics and Suricata alerts) and from a Kali terminal, execute the command

   `hping3 -S -flood <srv-ip-add>`

   which will generate SYN packets (`-S`), but without complete the 3-way handshake, and as fast as possible (`-flood`). This is a simulation of a DoS attack, known as **TCP SYN flood**, targeted to the server. While executing the command observe in the pfSense dashboard, the traffic volume, the CPU usage, and if the activity generated any alert. As before and in addition to recording the results obtained in the logbook, look to the `emerging-dos.rules` file content to justify the eventual alerts generated since it includes the rules (supposedly aiming) to detect that type of attack.

5. Next you will execute the command

   `hping3 -S -flood -rand-source <srv-ip-add>`

   which is the same but with an additional option to force Hping to use random values for source IP address. This time you should get some new alerts, but not associated with a DoS attack. It is an excellent opportunity to investigate the reason of those alerts, and discuss if the alerts correspond to the true nature of the activity.

6.2 Using Pytbull - optional, even more advanced task

For the next exercise, we will use **Pytbull**, a powerful public domain modular framework to test IDS/IPS, developed in python. It uses some other tools, such as **Nmap**, **Tcpreplay**, **Nikto**, **Ncrack**, and **Hping**, to simulate network-based attacks. We need to install it in the Kali VM, but most of the required tools are already there, in particular **Ncrack** – which, otherwise and

following Pytbull's documentation, would need to be compiled, what may be a difficult task in a strict Linux environment like Kali.

In fact there is a newer version you can use, Pytbull-ng. It is a fork of the original project, using a docker based architecture, updated tests, better performance, and easier implementation. Unfortunately, even being possible to install docker on Kali, it may be tricky[9], and you should think on using a dedicate machine to run Pytbull-ng instead the Kali Linux, or even a VM – that is why this guide keeps on utilising the original Pytbull.

Pytbull's basic operation consists of i) generate the traffic mimicking selected attacks, targeting a machine (supposedly a server) specified in the command line, eventually forcing the target to download files from external machines (depending on the type of test); ii) download the alert file generated by the IDS, from the target machine, through FTTP; and iii) check if each test was, or was not detected, and build a report.

After running the tests, Pytbull initiates an HTTP service at port 8080 in the local host, through which we can access the report and observe the IDS effectiveness. This feature assumes the IDS is running on the target machine, or otherwise it will not be possible to access the alert file. Our lab configuration (see Figure 2) does not match those requirements and so, to run Pytbull, we **need to create a fake empty alert file in the server (/var/log/suricata/-fast.log)**, which makes the Pytbull result analysis useless. **This is not an issue since we are not interested in the Pytbull report, but only in the traffic it generates** – however, Pytbull (or Pytbull-ng) is one of the most efficient tools to test an IDS and mastering it may be an important skill, in particular, because you can create or modify the tests crafted to your own purposes.

According to Pytbull documentation, in its last version (2016), it runs more than 300 tests divided into 11 classes, in several possible configurations. For our purpose the **stand-alone mode** is enough, dispensing the Pytbull server, essential only to simulate client-side attacks, which we will not deploy – see the documentation for additional information[10].

1. In our environment, installing **Pytbull** consists only on decompressing and placing it in the recommended location (`/opt/pytbull`). Anyway, it is advisable to follow the documentation instructions, but not trying to compile **Ncrack**, as referred above. Next, it is necessary to configure Pytbull, which is accomplished through the `config.cfg` file located in the `conf` directory, and using any text editor. The file has eight sections, and some of them require special attention:

   CLIENT where it is enough to enter the Kali VM's IP address and network interface linked to the internal network.

[9] See some instructions at https://www.kali.org/docs/containers/installing-docker-on-kali/.

[10] Additional documentation is available at https://andytanoko.wordpress.com/2016/06/30/pytbull-intrusion-detection-prevention-system/

PATH where we only need to uncomment the correct **alertsfile** field; take note of the path and file name since it is necessary to create it in the target machine (the server VM in your lab installation), as an empty file – you can use the `touch` command. Pytbull will try to get this file using FTP, and so the service must also be running – in the previous exercise we configured the server with vsftpd, which is adequate for the job.

ENV where the paths for all used tools must be correctly set. That can be hard since the paths may vary from one implementation to another. To find programs' path in Linux we can use any of the commands: i) `witch`; ii) `locate -b`; or iii) `type -p`. As a reference, Listing 1 presents a possible configuration for this section.

Listing 1: Pytbull configuration file

```
[ENV]
sudo              = /usr/bin/sudo
nmap              = /usr/bin/nmap
nikto             = /usr/share/nikto/nikto.pl
niktoconf         = /etc/nikto.conf
hping3            = /usr/sbin/hping3
tcpreplay         = /usr/bin/tcpreplay
ab                = /usr/bin/ab
ping              = /bin/ping
ncrack            = /usr/bin/ncrack
ncrackusers       = data/ncrack-users.txt
ncrackpasswords   = data/ncrack-passwords.txt
localhost         = 127.0.0.1
```

Even if we are running a small set of tests that do not require all tools, Pytbull checks it when starting and will stop in case of an error.

FTP where you need to enter the credentials to access the FTP server, as root (if you do not remember, with the CentOS and vsftpd, the username is root and the password is centos)

TESTS where we select any or all of the 11 test classes, setting a '`1`' for the ones to run, and a '`0`' for the others. There is no way of selecting individual tests in a class.

2. For the first attempt select only **ShellCodes**[11] tests, and from the directory where pytbull was installed (`/opt/pytbull`) run the command

   `./pytbull -t <srv-ip-addr>`

   while observing in background the pfSense dashboard, as before. When prompted, choose the first option to run a new campaign, and accept the aware notice. In case an error comes

[11] A SheelCode attack consists of embedding a piece of malware in a packet payload, aiming to have it executed in the target machine, and most likely, opening a shell that will let the attacker to gain access.

up, it is necessary to correct some configuration parameter in the configuration file – the Pytbull's documentation includes some comments on possible errors, too.

Pytbull will identify the individual tests as they are running. Register in your logbook the resulting alerts, as well as the system performance indicators (indicative values during the process). Go to the `Services / Suricata / Alerts` page to get more information about the alerts (priority, explored service, and alert class). Search on the web for more details pertaining to the ShellCode attack itself. Can you consider the registered alerts as true positives? Should you had received some more alerts? If yes, what is missing?

Finally, stop the Pytbull's web server execution pressing `Ctrl+C`.

3. Next, you will repeat the previous experience, but selecting **evasionTechniques** tests class. Pytbull will lunch several tests, using Nmap, Nikto, Icmp, and even Javascript, over TCP and UDP. This is a very reach group of tests that will produce a large number of alerts. Execute Pytbull using the same procedure as before, but it may be a good idea to clean alerts first. After finishing and stopping Pytbull, gp to the Suricata's alerts page and try to relate the alerts with the activity generated, always with a focus on the efficiency concerning false positives and true positives.

After completing the previous exercises, you will probably end up with thousands of alerts of different types and embedding an enormous amount of information, making almost impossible to extract useful knowledge to support effective security response decisions – unless you know precisely what you are looking for.

To address that difficulty, we need more than just pure Security Engineering skills. Given the number of variables and the amount of data available, we need to resort to some **data analysis skills**. However, the technological development in that area produced very complex frameworks and tools, supported on also sophisticated analysis methods, **forcing a long learning curve**, in particular when the data gets immensely big – mainly when dealing with large organisations and multiple log sources, well above the single NIDS we used in our simple lab. So, in real scenarios, it is probably more efficient to segregate the Security Engineering functions from the Data Analyst functions, despite being true they need both to work together for a better outcome.

The amount of data is more modest for small organisations, but it is almost certainly impossible to allocate enough human resources to follow the above strategy. In such cases, the person in charge needs to accomplish the job himself/herself, requiring the support of adequate analysis and visualisation tools.